

EE/CS 148B — Homework 3: Vision-Language Models

Spring 2026

1 Vision Transformer (§2)

Problem (vit_pooling): CLS token vs. mean pooling

For downstream tasks requiring spatial reasoning—counting objects, locating regions, or visual question answering that refers to specific image regions—**attention pooling** (or passing the full patch sequence) is expected to perform best. The CLS token aggregates global semantics into a single fixed-size vector, discarding which patch corresponds to which region of the image; mean pooling averages patch embeddings and preserves more distributed information but still loses the positional structure needed to answer relational questions (e.g., “What color is the cube to the left of the sphere?”). Condensing the entire image to one vector before the language model means the LM has no access to spatial layout, making it impossible to localize objects or reason about their relative positions.

Problem (vit_patch_size): Effect of patch size

Number of patches for 224×224 images:

Patch size P	$N = (224/P)^2$	Forward-pass time (ms)
8	784	38.10 ± 0.28
16	196	9.07 ± 0.27
32	49	8.18 ± 0.64

Measurements taken on an A100-SXM4-80GB GPU, batch size 16, ViT with $d_{\text{model}} = 384$, 6 heads, 6 blocks. Averaged over 20 steps after 5 warmup steps using `torch.cuda.synchronize()`.

Discussion. Self-attention compute scales as $O(N^2 d_{\text{model}})$, so shrinking P from 16 to 8 quadruples N (196→784) and roughly $16\times$ the attention compute. In practice, $P=8$ is $\sim 4\times$ slower than $P=16$ on this hardware (38 ms vs. 9 ms). $P=16$ and $P=32$ are similarly fast because the bottleneck shifts to MLP and projection layers rather than attention at these sequence lengths.

When to accept the $P=8$ trade-off. When spatial fine-grained reasoning is critical (medical imaging, OCR, dense visual QA), the extra compute of small patches is justified because higher-resolution feature maps enable the model to distinguish subtle spatial patterns that coarser patches would merge.

2 CLIP-Style Contrastive Pretraining (§3)

Problem (infonce): Symmetric InfoNCE

The symmetric InfoNCE loss is

$$\mathcal{L} = \frac{1}{2}(\text{CE}(S, y) + \text{CE}(S^\top, y)), \quad S = \text{image_embeds} \text{text_embeds}^\top \cdot e^{\text{logit_scale}}.$$

Why symmetric. The first term $\text{CE}(S, y)$ trains each image embedding to rank its paired caption above all other captions in the batch (image-to-text). The second term $\text{CE}(S^\top, y)$ trains each text embedding to rank its paired image above all other images (text-to-image). Averaging both directions makes the shared embedding space isotropic with respect to both modalities and ensures both encoders receive equal gradient signal.

Problem (clip_train): CLIP pretraining on EuroSAT

Hyperparameters: `img_size=64`, `patch_size=8`, $d_{\text{model}} = 384$, 6 heads, 6 blocks, batch size 256, $\text{lr} = 3 \times 10^{-4}$, AdamW (weight decay 0.1), cosine schedule with 200 warmup steps, 20 epochs.

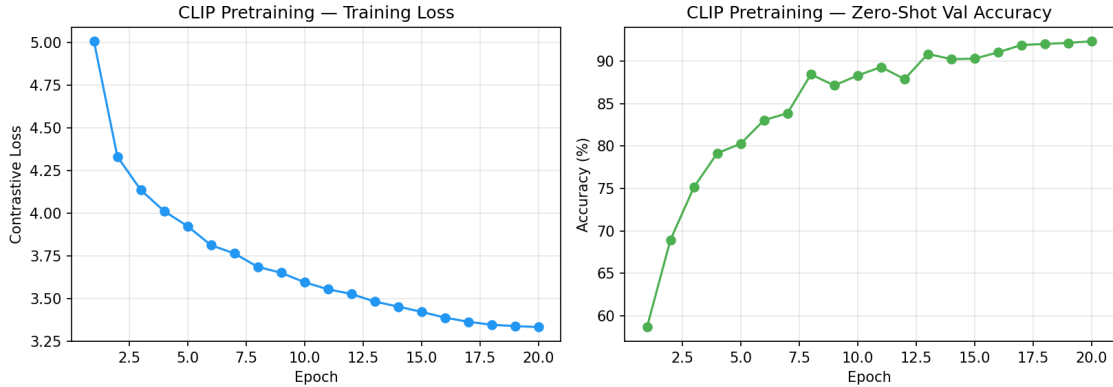


Figure 1: CLIP pretraining on EuroSAT: training contrastive loss (left) and zero-shot validation accuracy (right) vs. epoch.

Discussion. Training loss decreases steadily from 5.0 to 3.3 across 20 epochs, while zero-shot validation accuracy rises rapidly to $\sim 88\%$ in the first 8 epochs and then gains more slowly, reaching 92.3% at epoch 20. Training loss continues to improve even after validation accuracy plateaus above 90%, because the loss measures fine-grained pairwise similarity across all batch pairs (many of which share the same class-prompt caption in EuroSAT), whereas accuracy only measures coarse class-level assignment.

Problem (clip_zeroshot): Qualitative analysis

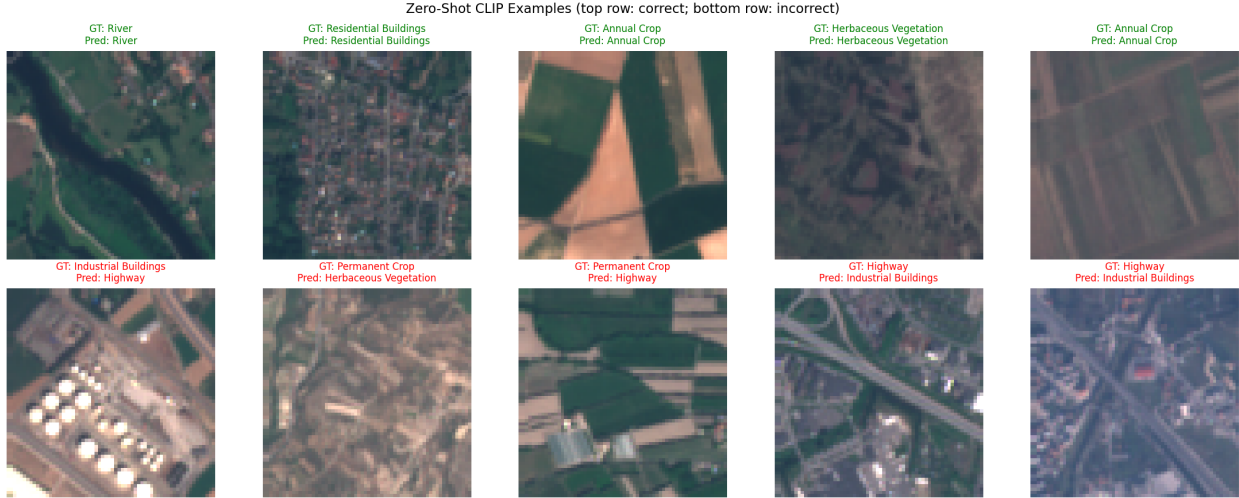


Figure 2: Ten zero-shot CLIP classification examples on the EuroSAT test set (top row: correct, green titles; bottom row: incorrect, red titles). Ground-truth (GT) and predicted class labels are shown for each image.

Five correctly classified and five incorrectly classified test images are shown above. Correct classifications include *River*, *Residential Buildings*, *Annual Crop*, and *Herbaceous Vegetation* — all visually distinctive at 64×64 . Incorrect predictions show systematic confusions: *Permanent Crop* is mistaken for *Herbaceous Vegetation* (both appear as fine-textured green patches) and for *Highway* (linear field boundaries resemble roads); *Highway* is confused with *Industrial Buildings* and vice versa (both show large grey paved areas from above). All mistakes involve visually similar categories, with no nonsensical errors observed.

This indicates the embedding space has learned meaningful semantic structure where nearby classes share visual appearance. Errors reflect genuine low-resolution ambiguity: at 64×64 , fine texture differences between crop types and the grey-paved appearance shared by roads and industrial sites are not reliably distinguishable.

3 LoRA Fine-Tuning (§4)

Problem (lora_linear): LoRA-wrapped linear layer

LoRA wraps an existing `nn.Linear` by freezing its weights and adding trainable low-rank matrices $A \in \mathbb{R}^{r \times d_{\text{in}}}$ (Kaiming-uniform init) and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ (zero init). The forward pass computes

$$W'x = Wx + \frac{\alpha}{r} BAx.$$

ViT with LoRA rank 8 parameter counts:

Metric	Value
Total parameters	10,737,792
Trainable parameters	258,048
Trainable ratio	2.40%

LoRA is applied to `q_proj` and `v_proj` in all 6 attention blocks (72 `LoRALinear` modules total).

Problem (`lora_compare`): Full FT vs. LoRA vs. linear probe

Starting from the CLIP-pretrained ViT, fine-tuned on RESISC45 for 10 epochs:

Method	Test Acc	Trainable Params	Peak GPU Mem	Wall-clock
Linear probe	37.84%	17,325	239 MB	71 s
LoRA ($r=8$)	41.67%	275,373	1,177 MB	171 s
Full FT	63.54%	10,755,117	1,719 MB	182 s

Discussion. Full fine-tuning achieves the best accuracy (63.5%) by adapting all 10.7M parameters. The linear probe is extremely memory-efficient (17K params, 239 MB peak) but lowest accuracy (37.8%), indicating frozen CLIP features are not perfectly aligned with the 45-class RESISC45 distribution. LoRA sits in between: with only 2.4% trainable parameters, it achieves 41.7%—a 3.9 pp improvement over the frozen probe at modest memory overhead. Full FT’s 22 pp advantage over LoRA likely stems from LoRA’s rank constraint limiting how much feature representations can shift for 45 new classes.

Problem (`lora_rank`): Rank sweep

Sweep over ranks $r \in \{1, 2, 4, 8, 16, 32, 64\}$ with $\alpha = 2r$ (so $\alpha/r = 2$):

Rank r	Test Acc	Trainable Params	Wall-clock
1	33.00%	49,581	110 s
2	35.71%	81,837	223 s
4	39.44%	146,349	143 s
8	41.67%	275,373	171 s
16	44.41%	533,421	127 s
32	46.86%	1,049,517	94 s
64	50.98%	2,081,709	98 s

Discussion. Accuracy increases monotonically from $r=1$ (33.0%) to $r=64$ (51.0%) with no clear saturation point: gains per doubling remain roughly 2–4 pp throughout the tested range, and the largest single jump (+4.1 pp) actually occurs at $r=32 \rightarrow 64$. Even $r=64$ trails full fine-tuning (63.5%) by 12.5 pp, because for this small ViT ($d_{\text{model}}=384$) the adapter is already $64/384 \approx 17\%$ of the feature dimension, far from the “low-rank” regime; in large-model fine-tuning the same rank represents $< 2\%$ of the feature dimension, so LoRA converges faster relative to full FT. The near-linear trend on a log-rank axis suggests the fine-tuning update is not strongly low-rank for this task.

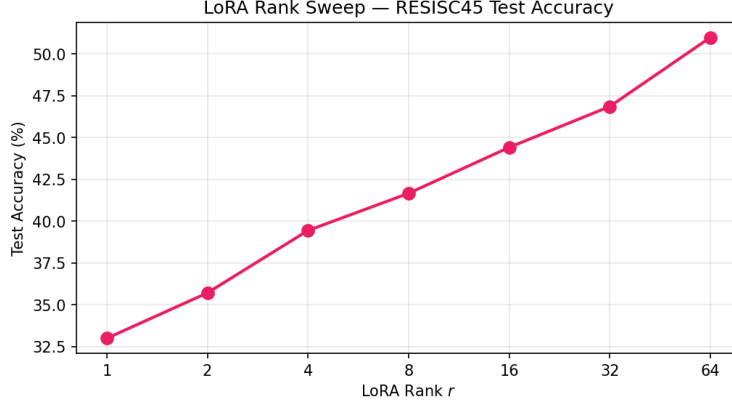


Figure 3: LoRA rank sweep on RESISC45: test accuracy vs. rank r ($\log_2$ scale).

4 Vision-Language Model (§5)

Problem (projector): Vision-language projector

The projector is a 2-layer MLP: $\text{Linear}(d_{\text{image}}, 4d_{\text{image}}) \rightarrow \text{GELU} \rightarrow \text{Linear}(4d_{\text{image}}, d_{\text{decoder}})$.

Why more than a single linear layer. A single linear projection performs only an affine map between image and decoder embedding spaces, which may be insufficient when both the vision encoder and decoder are frozen. The nonlinear hidden layer lets the projector learn a richer, task-adaptive mapping—a “translator” that reshapes image features into the semantic geometry expected by the language model’s embedding space. The $4\times$ hidden expansion adds capacity without a large parameter count.

Problem (masking): Image-block attention

(1) **Attention mask diagrams.** For 4 visual tokens ($v_1, \dots, v_4$) and 3 text tokens (t_1, t_2, t_3); ■ = attend, □ = blocked.

M1: Fully causal

	v_1	v_2	v_3	v_4	t_1	t_2	t_3
v_1	■	□	□	□	□	□	□
v_2	■	■	□	□	□	□	□
v_3	■	■	■	□	□	□	□
v_4	■	■	■	■	□	□	□
t_1	■	■	■	■	■	□	□
t_2	■	■	■	■	■	■	□
t_3	■	■	■	■	■	■	■

M2: Bidirectional inside image, causal elsewhere

	v_1	v_2	v_3	v_4	t_1	t_2	t_3
v_1-v_4	full bidirectional block				□	□	□
t_1	■	■	■	■	■	□	□
t_2	■	■	■	■	■	■	□
t_3	■	■	■	■	■	■	■

(2) Expected performance. M2 (bidirectional inside image) should perform better because it aligns with how the ViT originally processed patches via bidirectional attention. Under M1, patch v_i can only attend to $v_1, \dots, v_{i-1}$, imposing an artificial left-to-right ordering on a 2D grid with no inherent sequential structure. M2 lets each visual token attend to all other visual tokens, preserving full image context.

(3) Experimental results (500-step runs, all-patches injection):

Mask mode	Val exact-match accuracy
causal	34.8%
image_bidir	30.8%

Discussion. Contrary to the theoretical expectation, `image_bidir` (30.8%) underperforms `causal` (34.8%) at 500 steps. A likely explanation is that SmolLM2 was pretrained exclusively with causal attention; introducing a non-causal attention pattern for a block of tokens at the prefix disrupts the positional encoding assumptions baked into the pretrained weights. The model needs more steps to adapt to this novel masking regime, so the advantage of bidirectional image attention may only emerge with longer training or when the decoder is also fine-tuned.

Problem (injection_compare): Which injection strategy is best?

2000 training steps, freeze config A (projector only):

Strategy	Val Acc	Visual Tokens	Peak GPU Mem	Time/step
CLS only	37.0%	1	3,402 MB	77.2 ms
All patches	42.4%	65	6,689 MB	74.1 ms
Interleaved	37.0%	65	6,689 MB	91.4 ms

Timing: freeze config A (projector only), batch size 4, averaged over 20 steps including backward pass.

Discussion. All-patches injection (42.4%) outperforms both CLS-only (37.0%) and interleaved (37.0%). CLS discards spatial information by pooling the entire image into one token. Interleaved injection, despite using all 65 visual tokens, performs the same as CLS-only here — likely because the `<image>` placeholder is rare in the pretrained vocabulary and the model struggles to learn when to attend to interleaved visual tokens without fine-tuning the decoder. Direct prepending (all-patches) is the most natural format for a causal decoder seeing a prefix of visual tokens. Surprisingly, all-patches is slightly *faster* than CLS-only per step (74 vs. 77 ms) — the 65-token prefix is long enough to saturate GPU memory bandwidth, masking the extra compute, while CLS incurs more Python/CUDA overhead per token. Interleaved is the slowest (91 ms) due to the extra bookkeeping needed to splice visual tokens into arbitrary positions.

Problem (vlm_qualitative): What has your VLM learned?

Evaluated on 500 CLEVR-mini val examples with the best all-patches checkpoint (config A):

Question type	Accuracy
compare_attr	64.3%
exist	58.3%
spatial	43.6%
overall	42.4%
query_attr	35.4%
count	33.3%

The model handles yes/no questions best (*exist*, *compare_attr*: > 58%) because these require only a binary decision about global scene properties. *Count* questions are hardest (33.3%): counting objects in a 64×64 image requires precise spatial localization, which is challenging when the projector is the only trainable component. Example predictions:

Question (truncated)	Gold	Pred	
Are there fewer gray metal cylinders...?	yes	yes	✓
What number of things are left of...?	1	1	✓
What material is the red object?	rubber	rubber	✓
Is there anything else the same size as the brown thing?	yes	yes	✓
Is the large matte cylinder the same color as the shiny cube?	no	no	✓
How many things are either metal objects in front of the large purple cylinder or purple cylinders left of the brown thing?	4	1	×
Is there anything else that is the same shape as the small red shiny object?	no	yes	×
The metallic cylinder same size as the yellow sphere is what color?	purple	brown	×
What color is the tiny sphere?	cyan	brown	×
Does the blue rubber object have the same shape as the big metallic thing right of the matte cube?	no	yes	×

Discussion. The model handles binary questions (*exist*, *compare_attr*) best because the decoder can exploit language priors (defaulting to “yes” or a common attribute) with only a coarse visual signal. Counting errors (e.g., gold=4, pred=1) and color-attribute errors (e.g., gold=cyan/purple, pred=brown) are most likely *encoder* failures: at 64×64 with 65 patch tokens, fine-grained spatial relations and small-object colors are poorly resolved, so increasing the image resolution or patch granularity should help. Incorrect exist answers (e.g., gold=no, pred=yes) are more likely *decoder* failures driven by language priors, since the pretrained LM is biased toward “yes” for existence questions. To distinguish the two, one can run the same question through the text-only decoder (without any image): if it predicts the wrong answer, the fault is the language prior; if it is correct only when the image is provided, the fault is the encoder’s inability to encode the relevant visual detail.

Problem (freezing): What to train, and when

Config	Encoder	Projector	Decoder	Val Acc	Train Params	Peak Mem
A	frozen	trained	frozen	42.4%	2,067K	6,689 MB
B	frozen	trained	LoRA $r=8$	39.4%	2,886K	6,975 MB
C	frozen	trained	full FT	40.4%	363,888K	10,276 MB
D	full FT	trained	full FT	42.4%	374,626K	10,753 MB

Discussion. Config A (projector-only training) achieves 42.4% at minimal cost—just 2M trainable parameters and ~ 6.7 GB peak memory—suggesting the frozen CLIP encoder already produces rich visual features that the projector can bridge to the decoder with only 2,000 steps. Config B (decoder LoRA) performs *worse* than A (39.4% vs. 42.4%), likely because adding LoRA adapters to a bfloat16 decoder introduces dtype and gradient-flow noise that destabilizes training without long enough fine-tuning to overcome the frozen pretrained decoder’s priors. Config C (full decoder FT) achieves 40.4%, marginally below A: fully fine-tuning 363M decoder parameters on this small CLEVR dataset quickly overfits or shifts the decoder’s language priors away from what the projector produces, hurting performance within 2,000 steps. Config D (everything unfrozen) matches A at 42.4% but with $181\times$ more trainable parameters and $\sim 60\%$ more memory—the gains from end-to-end training cancel with overfitting at this step budget. The consistent message is that for small step budgets on small datasets, training only the projector is surprisingly competitive: the frozen CLIP encoder and pretrained LM provide strong inductive biases that do not need to be overwritten.

5 Positional Encodings and RoPE (§6)

Problem (rope_1d): 1D RoPE — norm preservation

RoPE applies a rotation to each pair (x_{2i}, x_{2i+1}) , an orthogonal transformation (rotation matrices have determinant 1 and preserve norms). Measured norm difference after applying RoPE:

$$\max |||x|| - ||\text{RoPE}(x)||| = 9.54 \times 10^{-7},$$

confirming norm preservation up to floating-point precision.

Problem (rope_vs_learned): RoPE vs. learned positional embeddings

Implementation. The ViT now supports three positional encoding modes via a `pos_enc` argument:

- **Learned PE** (default): a learnable $(1, N+1, d_{\text{model}})$ embedding added to the input.
- **RoPE-1D**: no learned PE; instead, RoPE is applied to Q and K inside each attention head. CLS receives position 0; patch i receives position $i+1$ in raster-scan order.
- **RoPE-2D**: the head dimension is split in half; the first half is rotated by the patch column index, the second by the row index. CLS receives $(0, 0)$.

For extrapolation, the ViT is trained at 64×64 (patch size 8, $8\times 8 = 64$ patches) and evaluated zero-shot at 96×96 ($12\times 12 = 144$ patches). Learned PE uses bilinear interpolation of the 8×8 embedding grid to 12×12 ; RoPE methods generalise naturally since frequencies are computed on-the-fly.

Results.

PE method	Val acc (64×64)	Test acc (96×96 extrap.)	Δ (extrap. drop)
Learned (bilinear interp.)	92.33%	81.70%	−10.6 pp
RoPE-1D	91.09%	77.83%	−13.3 pp
RoPE-2D	92.45%	80.53%	−11.9 pp

Discussion. At the training resolution, all three methods perform similarly (~ 91 – 92%). On extrapolation to 96×96 , learned PE with bilinear interpolation degrades the least (-10.6 pp), likely because bilinear resizing of the training grid is well-calibrated after 20 epochs. RoPE-2D extrapolates better than RoPE-1D (-11.9 vs. -13.3 pp): by encoding true (x, y) patch coordinates, 2D RoPE places the new 12×12 patches in a consistent spatial space, whereas 1D raster positions do not scale gracefully to a wider grid. Overall, RoPE does not clearly outperform learned PE for extrapolation in this setting, but 2D coordinates are a more principled inductive bias for image data.

Problem (rope_2d): 2D RoPE

Implementation. RoPE2D (in `basics/rope.py`) splits `head_dim` into two halves. The first half applies 1D RoPE using the patch column index; the second applies 1D RoPE using the row index. This is equivalent to independent rotations along x and y axes so that the attention dot product encodes the 2D *relative offset* $(\Delta x, \Delta y)$ between any two patches. The ViT is modified so that when `pos_enc="rope2d"`, learnable PE is removed and all attention blocks use RoPE2D via custom `RoPEBlock` layers.

Zero-shot accuracy on EuroSAT (64×64 training): **92.45%**; extrapolation at 96×96 : **80.53%**.

RoPE-2D matches learned PE in in-distribution accuracy and drops only 1.9 pp more on extrapolation. Because 2D RoPE encodes the correct spatial relationship between patches (column-row coordinates) rather than an arbitrary raster order, it provides a more principled representation for image data where the 2D arrangement of features is semantically meaningful.

Problem (mrope_written): Reasoning about M-RoPE

(1) Naive 1D positions for 64 patch tokens + CLS + 50 text tokens. With 64 patch tokens plus 1 CLS token prepended before a 50-token text prompt, text tokens receive position IDs 65–114. SmolLM2 handles sequences of hundreds or thousands of tokens, so out-of-distribution positions are not the primary concern at this scale. However, 2D structure is completely lost: the 64 patch tokens get sequential 1D positions (0–63) encoding raster-scan order through an 8×8 grid, but the model receives no (x, y) coordinates, preventing reasoning about spatial relationships like “the patch above” or “to the left.”

(2) First text token position under M-RoPE. The first text token receives position ($t=1, x=\text{grid_size}, y=\text{grid_size}$) e.g. (1, 8, 8) for an 8×8 patch grid. This is sensible because: (a) $t=1$ marks the first text token ($t=0$ for all image patches), and (b) x, y are set to `max_grid_coord + 1`, so text spatial IDs do not overlap with image IDs. A 10-token prompt after a 64-patch image starts at spatial position (8, 8) rather than 64, staying closer to positions the pretrained LM expects.

(3) Why three dimension chunks (t, x, y) rather than just (x, y) . M-RoPE splits the head dimension into three chunks for temporal, horizontal, and vertical coordinates. If only (x, y) were used, text tokens would have no dedicated way to encode sequential ordering—all text tokens would share the same (x, y) or encode order only in one spatial axis, confusing language modeling. The temporal t chunk is critical because language is inherently sequential, while image patches have no temporal ordering. Splitting into three chunks lets each coordinate type apply to the right tokens without interference.